



Animation



06. Animation

3D Animation

게임 프로그래머 | 이창진

| 목차

Chapter 1.

Assimp

Chapter 2.

Skeletal Mesh, Bone

Chapter 3.

Skeleton Influenced Bone Weight

Chapter 4.

Matrix Palette Update

Chapter 1.

Assimp

정의

Open Asset Import Library (assimp)

Open Asset Import Library is a library that loads various 3D file formats into a shared, in-memory format. It supports more than **40 file formats** for import and a growing selection of file formats for export.

3D 파일 포맷에 있는 데이터를 다룰 수 있는 라이브러리입니다.

```
std::unique_ptr<Assimp::Importer> importer;
```

```
m_>importer = std::make_unique<Assimp::Importer>();  
m_>importer->SetPropertyInteger(AI_CONFIG_PP_LBW_MAX_WEIGHTS, 4);  
std::string pathA = Utf8FromWString(pathW);  
m_>scene = m_>importer->ReadFile(pathA,  
aiProcess_Triangulate | aiProcess_JoinIdenticalVertices | aiProcess_ImproveCacheLocality |  
aiProcess_GenSmoothNormals | aiProcess_CalcTangentSpace |  
aiProcess_ConvertToLeftHanded |  
aiProcess_OptimizeMeshes | aiProcess_OptimizeGraph | aiProcess_LimitBoneWeights);
```

사용 예시

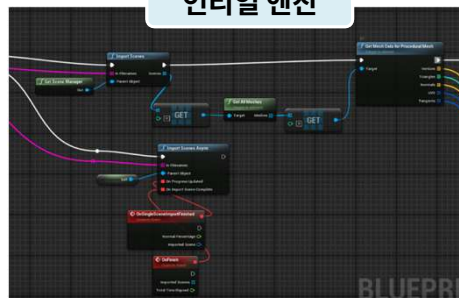
유니티

Trilib 2

Trilib is a cross-platform runtime 3D model importer for Unity

FBX, OBJ, GLTF2, STL, PLY, 3MF, DAE*, and ZIP files support
Cross-platform: Windows, Mac, Linux, UWP, Android, WebGL, and iOS
Import models from file-system, web, or any custom source
Mesh importing

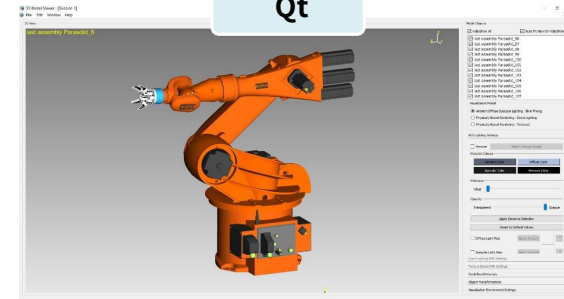
엔리얼 엔진



파이썬



Qt



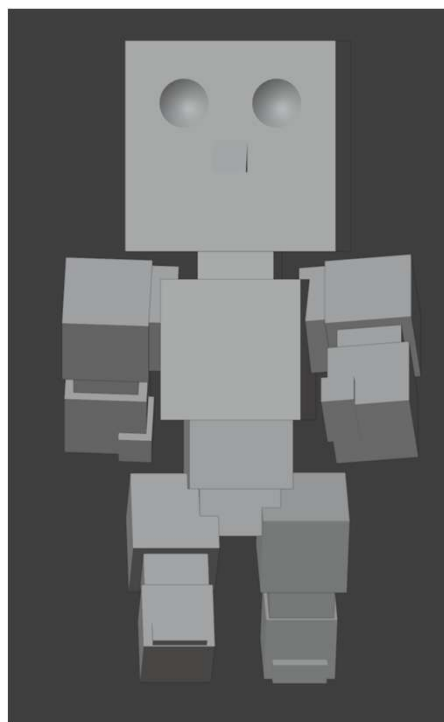
Skeletal Mesh

정의

2 개로 나뉩니다. 본이 있는 경우, 본이 없는 경우

1. 본이 있는 경우는 Skinned
2. 본이 없는 경우는 Rigid

모든 본은 부모 자식 관계로 연결되며 트리 형태로 데이터를 만들 수 있습니다. Root 본에 연결된 수많은 본들을 연결하여 가중치 값을 저장해 각 메쉬를 움직입니다.



Rigid



Skinned

Bone

Objects	1 / 3
Joints	408 / 496
Bones	204 / 248

Objects	3
Vertices	207,720
Edges	509,209
Faces	303,422
Triangles	303,422

정의

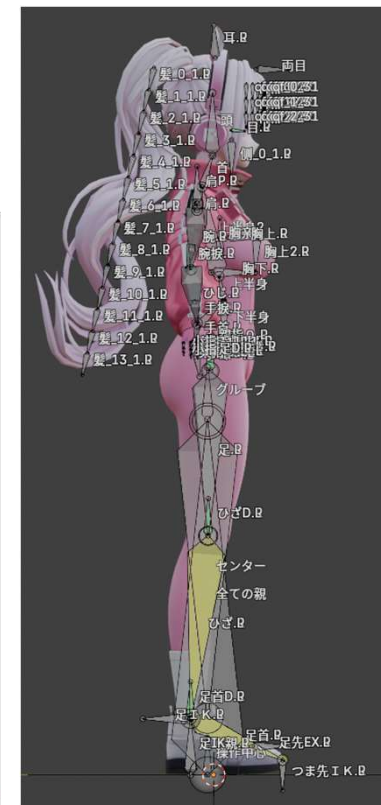
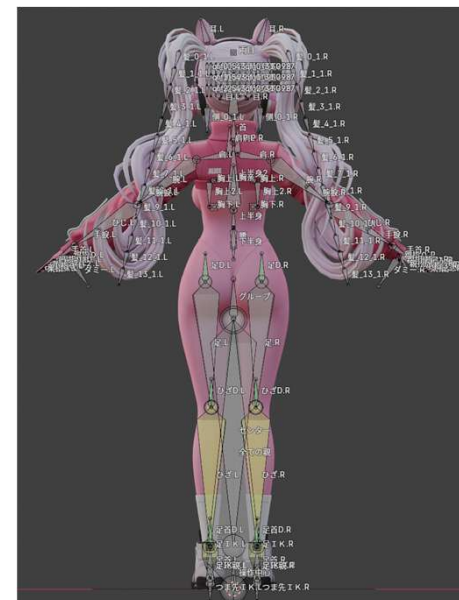
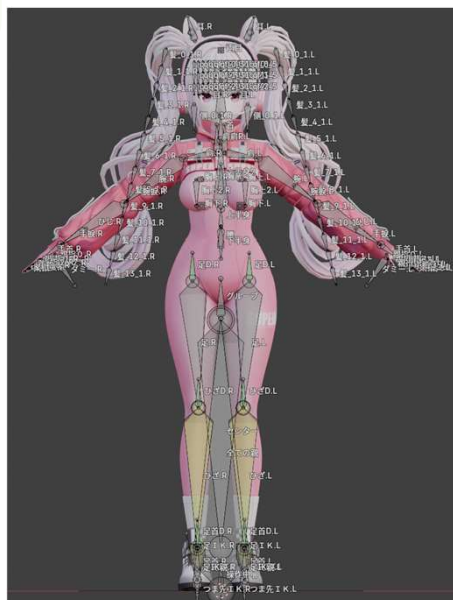
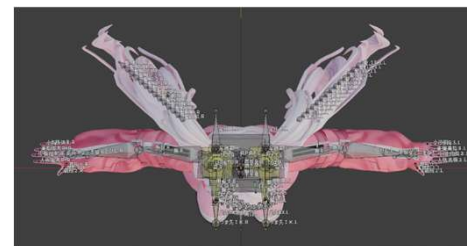
본 구조는 다음처럼 메시에 본이 들어 있습니다

모든 본은 부모 자식 관계로 연결되며 트리 형태로 데이터를 만들 수 있습니다. Root 본에 연결된 수많은 본들을 연결하여 가중치 값을 저장해 각 메쉬를 움직입니다.

오른쪽 모델에 대해

3D 캐릭터는 머리카락에도 본을심어 머리카락의 움직임도 만듭니다

Spine인 Root Bone을 기준으로 모든 본이 연결되어 있습니다. 본 개수는 248개이고, 버텍스 수는 207720개 입니다. 게임에 사용할 때는 버텍스 수를 2~3만개 이하로 줄여야 합니다. Decimate로 로우폴리로 만들고 노말맵을 적용하면 됩니다.



Bone Tree

설명

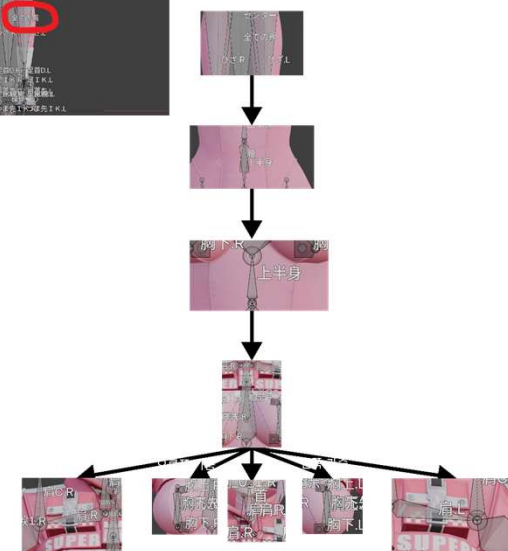
Root Bone 부터 트리 구조로 되어있습니다

Assimp로 가져온 값들 중 0번째 인덱스를 RootBone으로 두고 계산합니다

방향과 선후 관계가 있고 사이클이 없는 그래프이기 때문에 위상 정렬로 순회하며 각 인덱스의 정보를 저장합니다.

```
struct MeshEntry
{
    const aiNode* node;
    const aiMesh* mesh;
    uint32_t materialIndex;
    uint32_t vertexCount;
    uint32_t indexCount; // 삼각형만 집계(3의 배수)
    size_t vertexOffset;
    size_t indexOffset;
    size_t entryIndex;
};
```

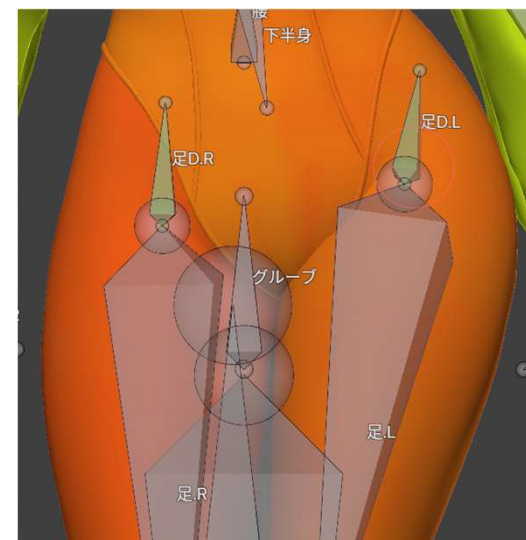
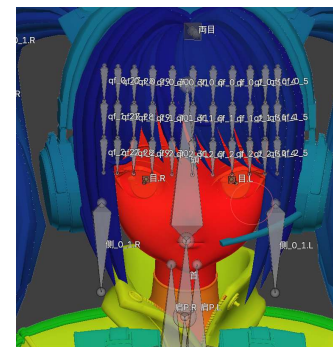
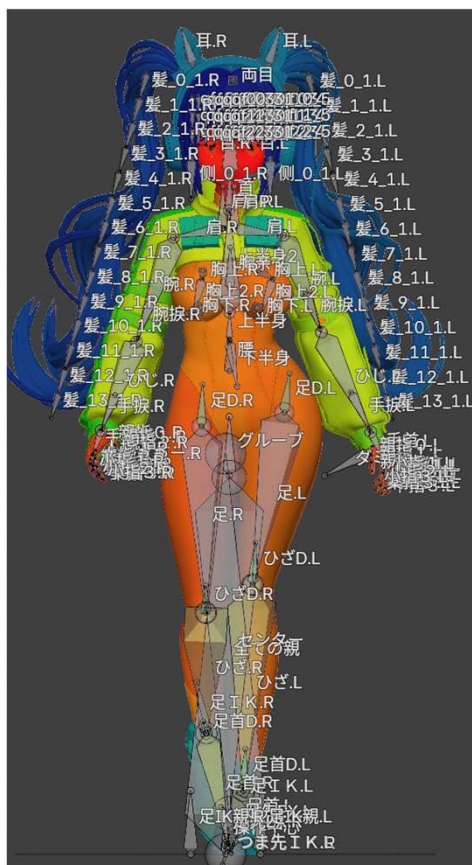
```
std::vector<MeshEntry> entries;
std::vector<std::pair<size_t, size_t>> levelRanges; // {start, count}
std::queue<const aiNode*> q;
q.push(scene->mRootNode);
size_t totalVertices = 0;
size_t totalIndices = 0;
while (!q.empty())
{
    size_t levelSize = q.size();
    size_t start = entries.size();
    for (size_t li = 0; li < levelSize; ++li)
    {
        const aiNode* node = q.front(); q.pop();
        for (unsigned mi = 0; mi < node->mNumMeshes; ++mi)
        {
            const aiMesh* mesh = scene->mMeshes[node->mMeshes[mi]];
            uint32_t vtx = mesh->mNumVertices;
            uint32_t idx = 0;
            for (unsigned f = 0; f < mesh->mNumFaces; ++f)
            {
                const aiFace& face = mesh->mFaces[f];
                if (face.mNumIndices == 3) idx += 3;
            }
            size_t entryIndex = entries.size();
            entries.push_back({ node, mesh, mesh->mMaterialIndex, vtx, idx, 0, 0, entryIndex });
            totalVertices += vtx;
            totalIndices += idx;
        }
        for (unsigned ci = 0; ci < node->mNumChildren; ++ci) q.push(node->mChildren[ci]);
    }
    size_t count = entries.size() - start;
    levelRanges.push_back({ start, count });
}
```



Bone Rigging

정의

버텍스의 각 점이 1개 이상의 본의 transform에 연결이 되고 각 본에 대한 가중치 값에 따라 글로벌 트랜스폼을 계산합니다. 버텍스에 대한 가중치를 누적합으로 만들어가며 사용합니다



Skeleton Build

설명

씬의 노드 트리를 BFS로 순회해서 key : value = 본 이름 : 본 인덱스 인 map 자료구조를 만들어 저장해둡니다



```
void FbxSkeleton::BuildFromScene(const aiScene* scene)
{
    m_Skeleton.clear(); m_NodeIndexOfName.clear(); m_RootIndex = -1;
    if (!scene || !scene->mRootNode) return;
    // 노드 수를 세어 미리 reserve (재할당 최소화)
    auto countNodes = [&](const aiNode* root){
        size_t cnt = 0; std::queue<const aiNode*> q; q.push(root);
        while (!q.empty()) { const aiNode* n = q.front(); q.pop(); ++cnt; for (unsigned i=0; i<n-
            >mNumChildren; ++i) q.push(n->mChildren[i]); }
        return cnt;
    };
    size_t nodeCount = countNodes(scene->mRootNode);
    m_Skeleton.reserve(nodeCount);
    m_NodeIndexOfName.reserve(nodeCount);

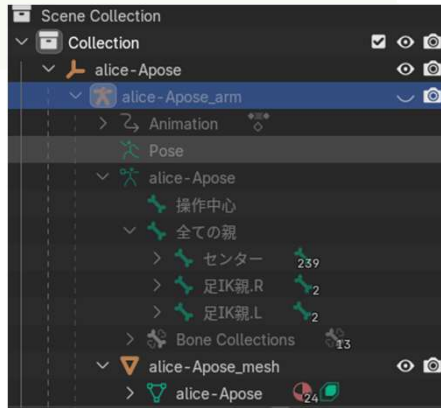
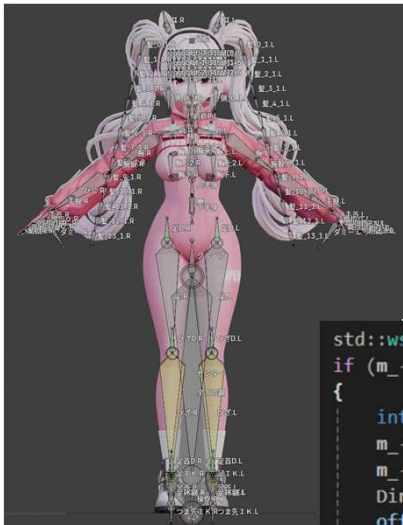
    // BFS로 부모→자식 순서로 생성
    std::queue<std::pair<const aiNode*, int>> q; // (node, parentIndex)
    q.push({ scene->mRootNode, -1 });
    int rootIndex = -1;
    while (!q.empty())
    {
        const aiNode* node = q.front().first; int parent = q.front().second; q.pop();
        int idx = (int)m_Skeleton.size();
        if (rootIndex < 0) rootIndex = idx;
        std::string nm = node->mName.C_Str();
        std::wstring nmW = WStringFromUtf8(nm);
        FbxSkeletonNode sn{}; sn.name = nm; sn.nameW = nmW; sn.parent = parent; sn.isBone = false;
        m_Skeleton.push_back(std::move(sn));
        m_NodeIndexOfName[m_Skeleton.back().name] = idx;
        if (parent >= 0) m_Skeleton[parent].children.push_back(idx);
        for (unsigned ci = 0; ci < node->mNumChildren; ++ci)
            q.push({ node->mChildren[ci], idx });
    }
    m_RootIndex = rootIndex;
}
```


Chapter 2

Skeleton CollectBones

설명

Mesh에 있는 Bone 이름을 수집해서
aiBone::mOffsetMatrix의 값들을 XMFLOAT4X4에
담아 저장해둡니다. Dx11에서 쓰는 데이터 구조를
사용하기 위함입니다.



```
std::wstring nameW = WStringFromUtf8(name);  
if (m_>BoneIndex.find(nameW) == m_>BoneIndex.end())  
{  
    int newIndex = m_>BoneIndex.size();  
    m_>BoneIndex[nameW] = newIndex;  
    m_>BoneNames.push_back(nameW);  
    DirectX::XMVECTOR offset = XMVECTOR();  
    off._11 = (float)b->mOffsetMatrix.a1; off._12 = (float)b->mOffsetMatrix.a2; off._13 = (float)b->mOffsetMatrix.a3; off._14 = (float)b->mOffsetMatrix.a4;  
    off._21 = (float)b->mOffsetMatrix.b1; off._22 = (float)b->mOffsetMatrix.b2; off._23 = (float)b->mOffsetMatrix.b3; off._24 = (float)b->mOffsetMatrix.b4;  
    off._31 = (float)b->mOffsetMatrix.c1; off._32 = (float)b->mOffsetMatrix.c2; off._33 = (float)b->mOffsetMatrix.c3; off._34 = (float)b->mOffsetMatrix.c4;  
    off._41 = (float)b->mOffsetMatrix.d1; off._42 = (float)b->mOffsetMatrix.d2; off._43 = (float)b->mOffsetMatrix.d3; off._44 = (float)b->mOffsetMatrix.d4;  
    m_BoneOffset.push_back(offset);  
    auto itNode = m_NodeIndex.find(name);  
    if (itNode != m_NodeIndex.end())  
    {  
        m_Skeleton[itNode->second].isBone = true;  
        if (m_Skeleton[itNode->second].nameW.empty()) m_Skeleton[itNode->second].nameW =  
            WStringFromUtf8(name);  
    }  
}
```

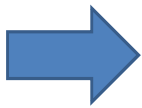
```
void FbxSkeleton::CollectBonesAndOffsets(const aiScene* scene)  
{  
    m_BoneNames.clear(); m_BoneOffset.clear();  
    if (!scene) return;  
    // 상한치 기반 reserve로 리해시/재할당 최소화  
    size_t totalBonesUpper = 0;  
    for (unsigned mi = 0; mi < scene->mNumMeshes; ++mi) totalBonesUpper += scene->mMeshes[mi]->mNumBones;  
    std::unordered_map<std::string,int> boneIndexofName; boneIndexofName.reserve(totalBonesUpper);  
    m_BoneNames.reserve(totalBonesUpper); m_BoneOffset.reserve(totalBonesUpper);  
    for (unsigned mi = 0; mi < scene->mNumMeshes; ++mi)  
    {  
        const aiMesh* mesh = scene->mMeshes[mi];  
        for (unsigned bi = 0; bi < mesh->mNumBones; ++bi)  
        {  
            const aiBone* b = mesh->mBones[bi];  
            std::string name = b->mName.C_Str();  
            if (boneIndexofName.find(name) == boneIndexofName.end())  
            {  
                int newIndex = (int)m_BoneNames.size();  
                boneIndexofName.emplace(name, newIndex);  
                m_BoneNames.push_back(name);  
                DirectX::XMVECTOR off;  
                off._11 = (float)b->mOffsetMatrix.a1; off._12 = (float)b->mOffsetMatrix.a2; off._13 = (float)b->mOffsetMatrix.a3; off._14 = (float)b->mOffsetMatrix.a4;  
                off._21 = (float)b->mOffsetMatrix.b1; off._22 = (float)b->mOffsetMatrix.b2; off._23 = (float)b->mOffsetMatrix.b3; off._24 = (float)b->mOffsetMatrix.b4;  
                off._31 = (float)b->mOffsetMatrix.c1; off._32 = (float)b->mOffsetMatrix.c2; off._33 = (float)b->mOffsetMatrix.c3; off._34 = (float)b->mOffsetMatrix.c4;  
                off._41 = (float)b->mOffsetMatrix.d1; off._42 = (float)b->mOffsetMatrix.d2; off._43 = (float)b->mOffsetMatrix.d3; off._44 = (float)b->mOffsetMatrix.d4;  
                m_BoneOffset.push_back(off);  
                auto itNode = m_NodeIndex.find(name);  
                if (itNode != m_NodeIndex.end())  
                {  
                    m_Skeleton[itNode->second].isBone = true;  
                    if (m_Skeleton[itNode->second].nameW.empty()) m_Skeleton[itNode->second].nameW =  
                        WStringFromUtf8(name);  
                }  
            }  
        }  
    }  
}
```

Skeleton BuildVertexTable

설명

노드 트리를 순회하면서 각 Mesh의 버텍스 버퍼에서의 시작하는 오프셋을 저장해둡니다.

메시가 시작하는 오프셋이란 각 메시가 배열에서 시작하는 인덱스입니다. 메시의 로컬 정점 인덱스를 글로벌 버텍스 버퍼 인덱스로 바꿀 때 사용합니다



```
struct Influence4 { unsigned short idx[4] = {0,0,0,0}; float w[4] = {0,0,0,0}; };
std::vector<Influence4> inf; inf.assign(verts.size(), {});
// Build base vertex table (mesh -> start in aggregated array) using same traversal order as geometry
std::vector<size_t> baseVertex; baseVertex.resize(m_>scene->mNumMeshes, 0);
size_t cursor = 0;
std::function<void(const aiNode*)> fillBase = [&](const aiNode* node){
for (unsigned mi = 0; mi < node->mNumMeshes; ++mi)
{
    unsigned meshIdx = node->mMeshes[mi];
    baseVertex[meshIdx] = cursor;
    cursor += m_>scene->mMeshes[meshIdx]->mNumVertices;
}
for (unsigned ci = 0; ci < node->mNumChildren; ++ci) fillBase(node->mChildren[ci]);
};
fillBase(m_>scene->mRootNode);
```

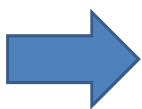
```
for (unsigned mi = 0; mi < m_>scene->mNumMeshes; ++mi)
{
    const aiMesh* mesh = m_>scene->mMeshes[mi];
    size_t base = baseVertex[mi];
    for (unsigned bi = 0; bi < mesh->mNumBones; ++bi)
    {
        const aiBone* b = mesh->mBones[bi];
        int boneIdx = -1;
        auto itB = boneIndexofBoneName.find(b->mName.C_Str());
        if (itB != boneIndexofBoneName.end()) boneIdx = itB->second;
        if (boneIdx < 0) continue;
        for (unsigned wi = 0; wi < b->mNumWeights; ++wi)
        {
            const aiVertexWeight& vw = b->mWeights[wi];
            size_t v = base + (size_t)vw.mVertexId;
            if (v >= inf.size()) continue;
            int slot = 0; float minW = inf[v].w[0];
            for (int s = 1; s < 4; ++s) { if (inf[v].w[s] < minW) { minW = inf[v].w[s]; slot = s; } }
            inf[v].idx[slot] = (unsigned short)boneIdx;
            inf[v].w[slot] = (float)vw.mWeight;
        }
    }
}
```

Skeleton Influences Weights

설명

각 본의 aiVertexWeight를 VB 인덱스로 변환해
버텍스별로 영향받은 상위 4개만 저장합니다

4개만 보관하는 이유는 Assimp에서도 최대 본
가중치를 4로 제한해두었기 때문입니다.



```
for (unsigned mi = 0; mi < m_>scene->mNumMeshes; ++mi)
{
    const aiMesh* mesh = m_>scene->mMeshes[mi];
    size_t base = baseVertex[mi];
    for (unsigned bi = 0; bi < mesh->mNumBones; ++bi)
    {
        const aiBone* b = mesh->mBones[bi];
        int boneldx = -1;
        auto itB = boneIndexofBoneName.find(b->mName.C_Str());
        if (itB != boneIndexofBoneName.end()) boneldx = itB->second;
        if (boneldx < 0) continue;
        for (unsigned wi = 0; wi < b->mNumWeights; ++wi)
        {
            const aiVertexWeight& vw = b->mWeights[wi];
            size_t v = base + (size_t)vw.mVertexId;
            if (v >= inf.size()) continue;
            int slot = 0; float minW = inf[v].w[0];
            for (int s = 1; s < 4; ++s) { if (inf[v].w[s] < minW) { minW = inf[v].w[s]; slot = s; } }
            inf[v].idx[slot] = (unsigned short)boneldx;
            inf[v].w[slot] = (float)vw.mWeight;
        }
    }
}
```

Skeleton NormalizeInfluences

문제상황

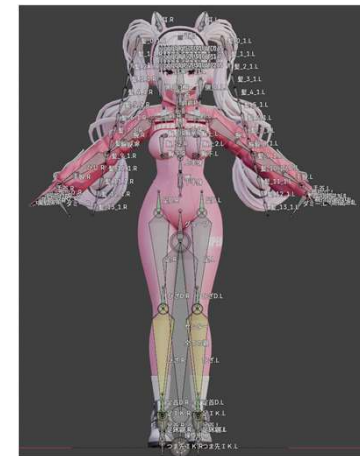
가중치의 합을 그대로 사용하니 애니메이션을 실행시켰을 때 메시의 부분들이 이상하게 커지거나 줄어드는 문제가 발생했습니다

시도 했던 것과 해결 방안

비슷한 아티팩트를 찾아보니 가중치의 합을 1로 정규화를 해야 애니메이션 실행 시 스케일이 정상적으로 작동한다는 것을 알게 되었습니다.

비가중치 버텍스는 본0에 1을 할당하고 스킴닝 여부를 결정합니다. 문제를 해결하고자 영향을 받은 버텍스들의 정규화를 도입했습니다

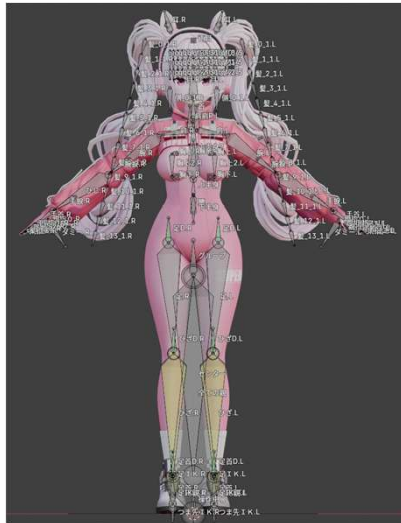
```
// Normalize and apply to vertices
for (size_t i = 0; i < inf.size(); ++i)
{
    float s = inf[i].w[0] + inf[i].w[1] + inf[i].w[2] + inf[i].w[3];
    if (s > 1e-6f) { float inv = 1.0f / s; for (int k = 0; k < 4; ++k) inf[i].w[k] *= inv; }
    verts[i].boneIdx[0] = inf[i].idx[0];
    verts[i].boneIdx[1] = inf[i].idx[1];
    verts[i].boneIdx[2] = inf[i].idx[2];
    verts[i].boneIdx[3] = inf[i].idx[3];
    verts[i].boneWeight = { inf[i].w[0], inf[i].w[1], inf[i].w[2], inf[i].w[3] };
}
m_ -> geometry.RebuildVBFromCPU(device);
```



Skeleton InitMetadata

설명

aiAnimation 배열을 for문으로 돌면서 이름, 길이(초), TicksPerSecond를 기록합니다



```
void FbxAnimation::InitMetadata(const aiScene* scene)
{
    m_Names.clear(); m_DurationSec.clear(); m_TicksPerSec.clear();
    if (!scene) return;
    if (scene->mNumAnimations == 0) return;
    m_Names.reserve(scene->mNumAnimations);
    m_DurationSec.reserve(scene->mNumAnimations);
    m_TicksPerSec.reserve(scene->mNumAnimations);
    for (unsigned i = 0; i < scene->mNumAnimations; ++i)
    {
        const aiAnimation* a = scene->mAnimations[i];
        std::string nm = a->mName.length > 0 ? std::string(a->mName.C_Str()) :
            (std::string("Anim") + std::to_string(i));
        double tps = (a->mTicksPerSecond != 0.0) ? a->mTicksPerSecond : 25.0;
        double durSec = (tps != 0.0) ? (a->mDuration / tps) : 0.0;
        m_Names.push_back(nm);
        m_TicksPerSec.push_back(tps);
        m_DurationSec.push_back(durSec);
    }
    m_Current = 0; m_TimeSec = 0.0; m_Playing = false;
}
```

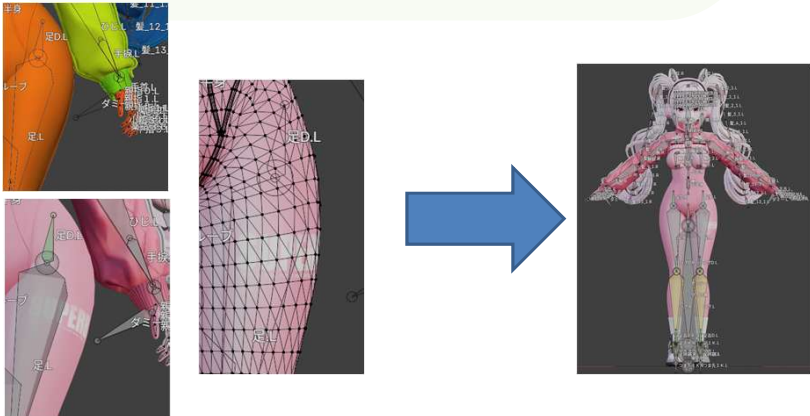
Matrix Palette Update

설명

시간에서의 포즈 변환 행렬을 만들고 각본의 스키닝 행렬을 만듭니다. 그리고 이를 palett에 저장합니다. 그리고 본들은 이전에 만든 노드의 글로벌 행렬을 사용해 다시 본 행렬을 만들어냅니다.

aiBone의 오프셋과 루트 역행렬을 구합니다.
 $Gi * G * Offset$

다음으로 이 Palette를 버텍스 셰이더에게 상수버퍼로 전달합니다. 만들어 놓은 본 행렬 배열과, 정점의 본 인덱스, 가중치 행렬 등의 모든 정보를 전달합니다.



```
XMMATRIX Gi = XMLoadFloat4x4(giPtr);
for (size_t bi = 0; bi < bones->size(); ++bi)
{
    auto itN = nodeMap.find((*bones)[bi]); if (itN == nodeMap.end()) continue;
    int nodeIdx = itN->second; if (nodeIdx < 0 || nodeIdx >= (int)m_GlobalScratch.size()) continue;
    XMMATRIX G = XMLoadFloat4x4(&m_GlobalScratch[(size_t)nodeIdx]);
    XMMATRIX Off = XMLoadFloat4x4(&(*offsets)[bi]);
    m_PaletteScratch[bi] = XMMatrixMultiply(XMMatrixMultiply(Gi, G), Off);
}
```

```
void FbxAnimation::UploadPalette(ID3D11DeviceContext* ctx, const std::vector<XMMATRIX>& pal)
{
    if (!ctx) return;
    Microsoft::WRL::ComPtr<ID3D11Device> dev; ctx->GetDevice(dev.GetAddressOf());
    EnsureBoneCB(dev.Get(), (int)pal.size() + 1);
    if (!m_pBoneCB) return;
    struct BoneCB { XMFLOAT4X4 m[1023]; unsigned int boneCount; float pad[3]; };
    BoneCB cb{}; size_t n = std::min(pal.size(), (size_t)1023);
    XMMATRIX I = XMMatrixIdentity();
    for (size_t i = 0; i < 1023; ++i) XMStoreFloat4x4(&cb.m[i], XMMatrixTranspose(I));
    for (size_t i = 0; i < n; ++i) XMStoreFloat4x4(&cb.m[i], XMMatrixTranspose(pal[i]));
    cb.boneCount = (unsigned int)n;
    D3D11_MAPPED_SUBRESOURCE mapped{};
    if (SUCCEEDED(ctx->Map(m_pBoneCB, 0, D3D11_MAP_WRITE_DISCARD, 0, &mapped)))
    { memcpy(mapped.pData, &cb, sizeof(BoneCB)); ctx->Unmap(m_pBoneCB, 0); }
}
```

감사합니다.